RESULTLANG

Compiler Architecture and IR Design

Presentation 2

CSE 303 - Compiler Design
Military Institute of Science and Technology

Md. Mahdi Israk Mahir | 202414016

Rafsan Jony Asif | 202414019

Prof. Dr. Kazy Noor E Alam Siddiquee

Presentation 2 brief

The assignment evaluates the design choices that connect source syntax to executable code.

01	**Compiler architecture**	Phase boundaries, artifacts and control flow
02	**Lexer and parser design**	Token rules, grammar and AST construction
03	**Semantic analysis**	Types, symbol table and diagnostic gates
04	**IR strategy**	Three Address Code, temporaries and labels

ResultLang language surface

SOURCE

```
student "Rahim";

math = 85;
english = 78;
science = 90;

total = math + english + science;
average = total / 3;

if (average >= 80)
    grade = "A+";
else
    grade = "B";

show(grade);
```

Compilation contract

- AST structure
- Symbol table
- **Three Address Code**
- Generated C
- Executable output

Assignments infer types. Comparisons produce Boolean values. show(...) sends a typed expression to the output stage.

Compiler architecture

FRONT END

1. **Source reader**
2. Lexical analysis
3. Parsing and AST
4. Semantic checks

MIDDLE END

1. **Typed AST**
2. Symbol table
3. TAC generation
4. Error gate

BACK END

1. **C code generation**
2. GCC compilation
3. program.exe
4. Terminal output

Fatal diagnostics block every downstream phase.

Project module map


```
src/
  lexer.l
  parser.y
  ast.c / ast.h
  symbol_table.c / .h
  semantic.c / .h
  tac.c / tac.h
  codegen.c / .h
  main.c

scripts/
  build.bat
  compile_file.bat
  run_demo.bat
```

lexer.l	Characters to tokens
parser.y + ast	Tokens to syntax tree
semantic + symbols	Meaning and type validation
tac.c	Typed tree to IR
codegen.c	IR or AST to C
main.c	Pipeline driver

Lexer design

Rule order

1. Skip whitespace and comments
2. **Match keywords before identifiers**
3. **Match >=, <=, ==, != first**
4. Capture numbers and strings
5. Reject unknown characters

Token classes

Keywords

`student` `if` `else` `show` `read`

Payloads

`ID` `NUMBER` `STRING` `TRUE` `FALSE`

Operators

`+` `-` `*` `/` `=` `>=` `<=` `==` `!=`

Delimiters

`(` `)` `{` `}` `;`

Line tracking: yylineno supports phase-specific diagnostics.

Tokenization example


```
if (average >= 80) grade = "A+";
```

IF	LPAREN	ID	GE	NUMBER
if	(	average	>=	80

RPAREN	ID	ASSIGN	STRING	SEMI
)	grade	=	A+	;

Longest match keeps >= as one token. Each payload retains its line number and value.

Parser and grammar design

CORE GRAMMAR

```
program  : student_decl stmt_list ;

student_decl
         : STUDENT STRING ';' ;

stmt     : ID '=' expr ';'
         | SHOW '(' expr ')' ';'
         | IF '(' expr ')' stmt ELSE stmt
         | '{' stmt_list '}' ;

expr     : NUMBER | STRING | TRUE | FALSE
         | ID | '(' expr ')'
         | '-' expr
         | expr '*' expr | expr '/' expr
         | expr '+' expr | expr '-' expr
         | expr GE expr | expr EQ expr ;
```

Precedence

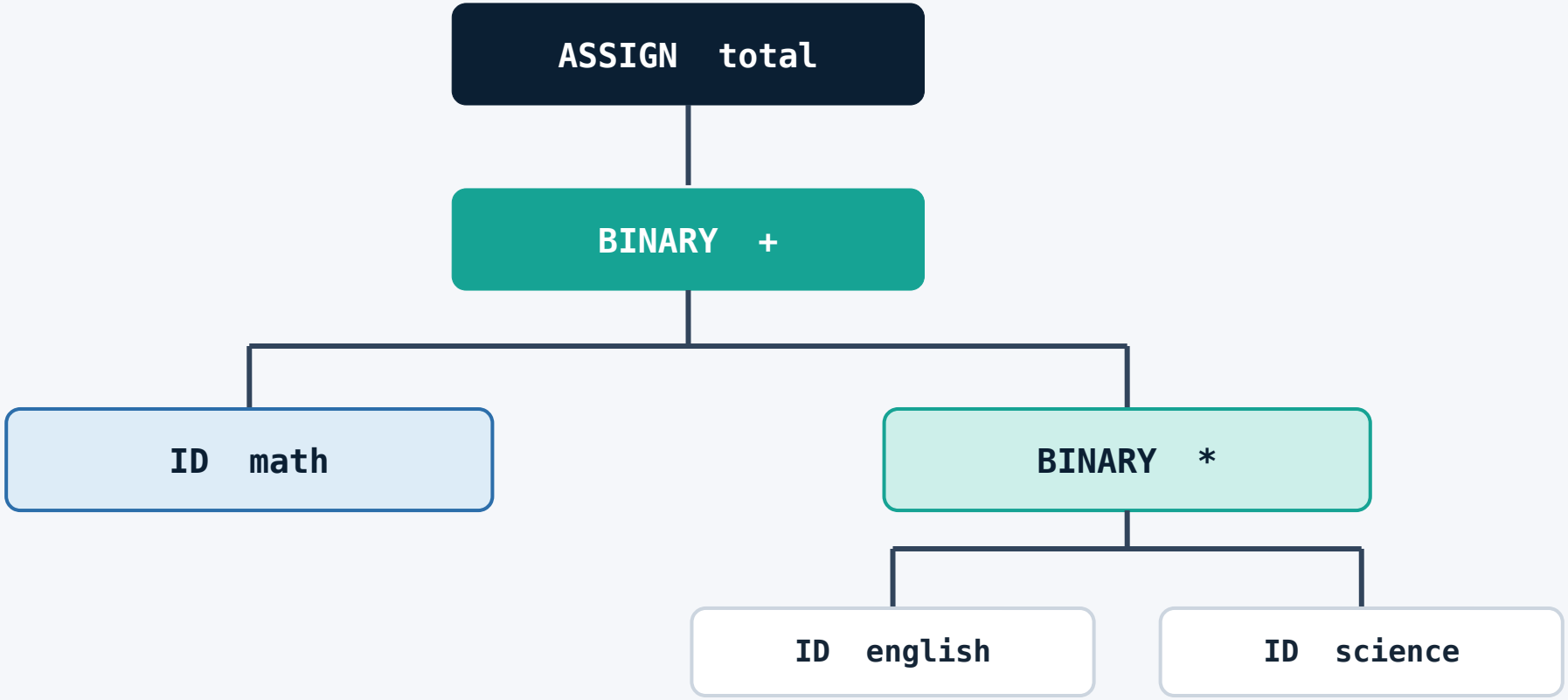
Highest	( ) literals ID
Unary	-expr
Multiply	* /
Add	+ -
Compare	> < >= <= == !=

Grammar actions create AST nodes. Semantic work occurs after parsing.

AST design


```
total = math + english * science;
```

The AST preserves meaning and precedence without punctuation nodes.



Semantic analysis and symbol table

Rules applied to the AST

Arithmetic	Both operands must be numbers
Comparison	Relational operands must be numbers
Assignment	A variable keeps one compatible type
Identifier	A value must exist before use
Condition	if requires a Boolean expression
Division	Constant zero is rejected

Symbol table after analysis

Name	Type	First line
student	string	1
math	number	3
total	number	6
average	number	7
grade	string	10

Fatal semantic errors stop TAC and C generation.

Error handling and phase gates

LEXER

Unknown character or malformed token

```
[lexical error] line 3: unknown character '@'
```

PARSER

Missing delimiter or invalid statement

```
[syntax error] line 5: expected ';'
```

SEMANTIC

Undefined name or incompatible type

```
[semantic error] line 7: undefined variable 'total'
```

Any fatal error sets a failure state. Later compiler phases do not run.

Three Address Code strategy

Design choices

- **One operation** — Each instruction contains one main operator
- **Temporaries** — t1, t2 and t3 hold partial results
- **Labels** — L1 and L2 represent control-flow targets
- **Typed input** — read.string and read.number retain intent
- **Readable calls** — param x followed by call show

EXAMPLE TAC

```
t1 = math + english
t2 = t1 + science
total = t2
t3 = total / 3
average = t3
t4 = average >= 80
ifFalse t4 goto L1
grade = "A+"
goto L2
L1:
grade = "B"
L2:
param grade
call show
```

If/else lowering

RESULTLANG

```
if (average >= 80)
    grade = "A+";
else
    grade = "B";
```

TAC

```
t1 = average >= 80
ifFalse t1 goto L1
grade = "A+"
goto L2
L1:
grade = "B"
L2:
```

1 **Evaluate condition**

2 **Branch to L1 when false**

3 **Join both paths at L2**

Labels preserve control flow without embedding target-machine details.

Target C and executable path



Generated C responsibilities

1. Declare typed variables
2. Preserve arithmetic and branches
3. Print ResultLang values
4. Protect runtime division and input

Implementation status

The supplied workflow identifies src/main.c as the verified runnable compiler driver. Separate lexer, parser and module files document the intended architecture. Presentation 2 therefore explains the design and does not claim a verified generated Flex/Bison build.

Next milestone: implementation evidence, executable generation and full testing.